Diagramme entité-relation de la base de données

Date : 2026-10-03 **Référence :** §II.C “Relations entre les tables” du manuel de certification. **Source de vérité :** `crates/pos-db/src/schema.rs`, `schema_hubrise.rs` et `schema_catalog.rs`, assemblés par `crates/pos-db/src/migrate.rs` (le SQL définit le schéma exact ; ce document reflète la révision actuelle : migrations 1 à 10, puis 20 à 25).

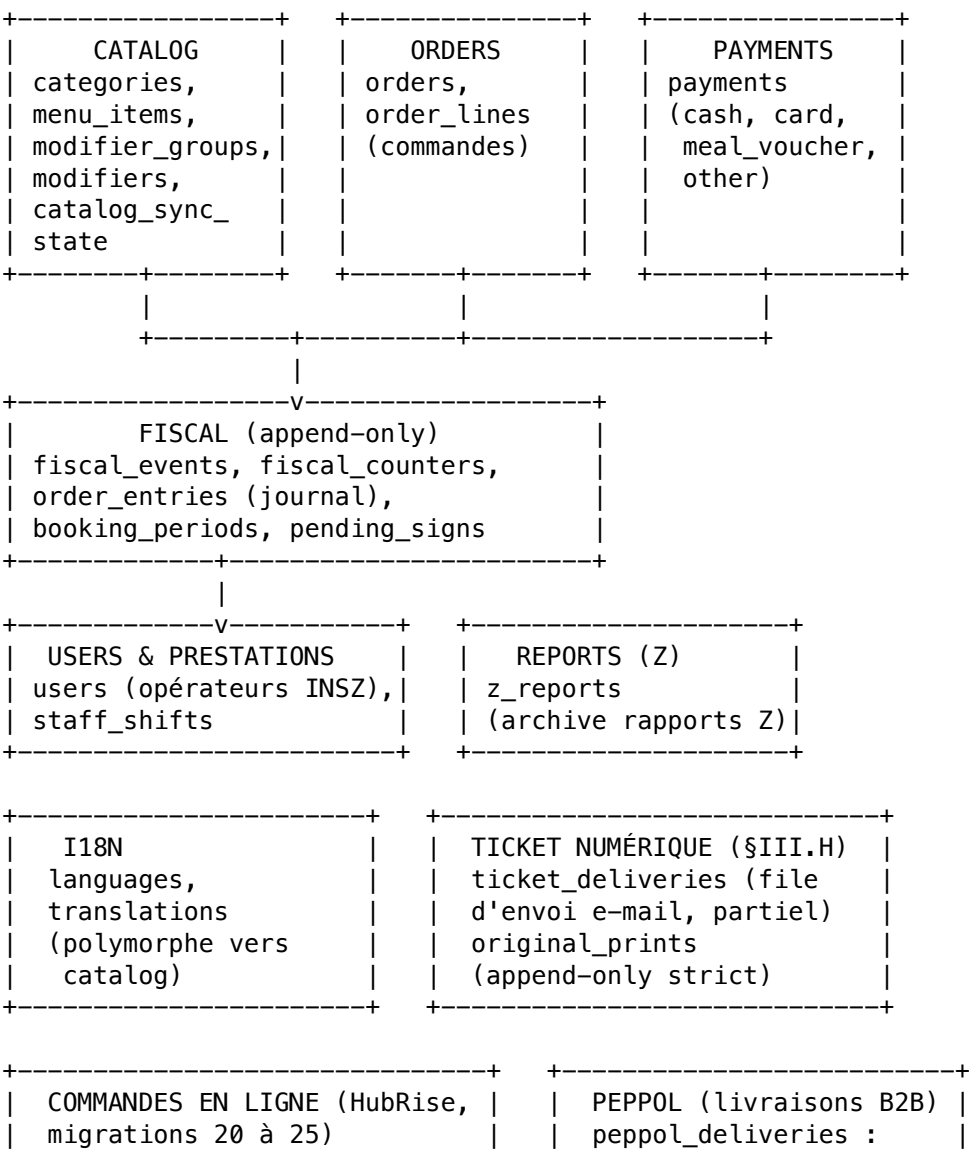
Ce document décrit visuellement les relations entre les tables de la base SQLite `pos.db`. Pour l’inventaire détaillé champ-par-champ, *voir : SCHEMA_BD_FR.md*. Pour la sécurité et les contraintes, *voir : SECURITE_BD_FR.md*.

Une note sur les clés : les colonnes `user_insz` (dans `orders`, `fiscal_events`, `pending_signs`, `staff_shifts`) contiennent l’INSZ de l’opérateur mais ne portent **aucune clé étrangère** vers `users` : un événement fiscal garde son INSZ même si l’utilisateur est désactivé ou retiré. Le lien est applicatif.

---

1. Vue d’ensemble par domaines

La base est organisée en 10 domaines logiques :



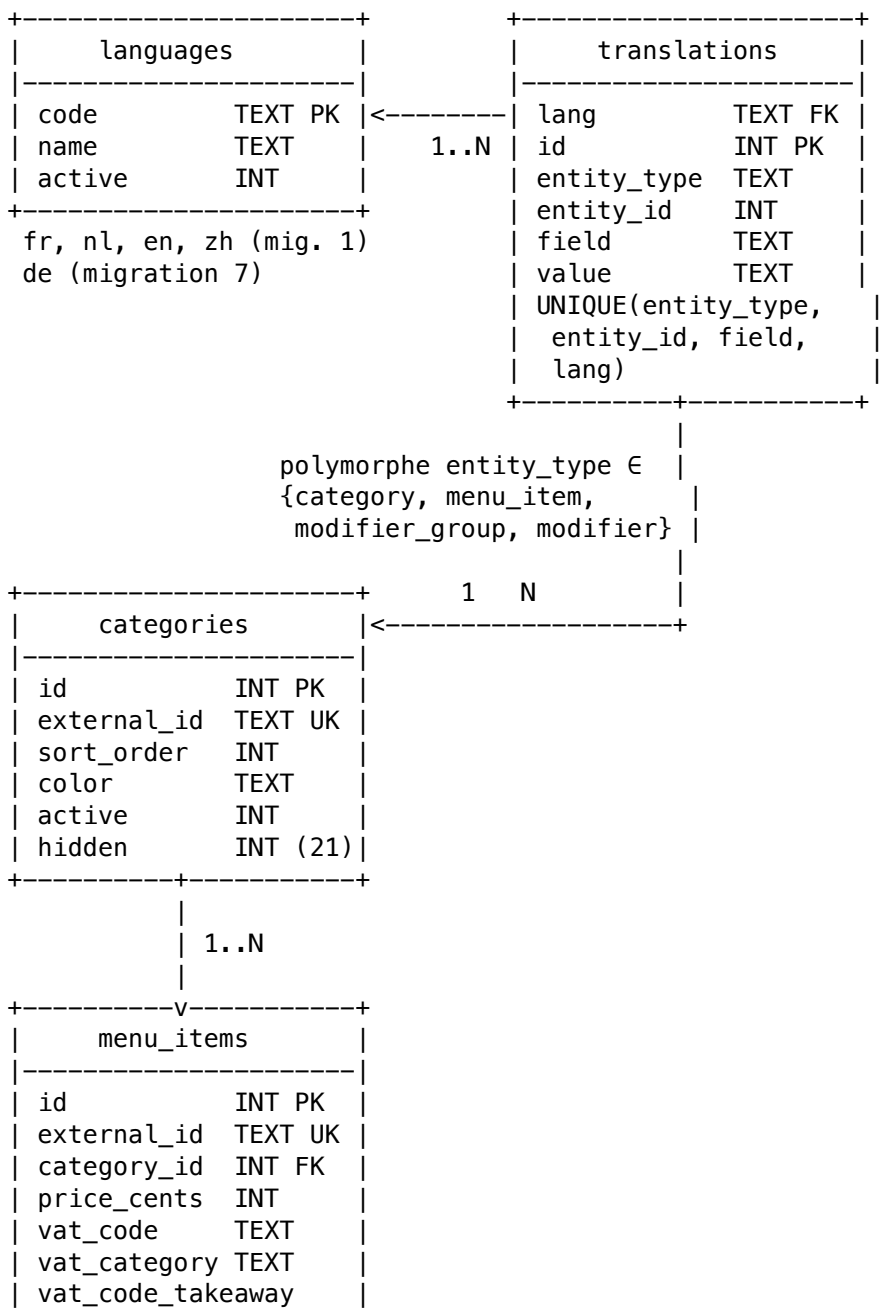
```
|  webshop_orders,              |   |  table présente, jamais  |
|  hubrise_channels,            |   |  écrite par la build     |
|  hubrise_cash_payment_refs,   |   |  cert (hors v1.0.0)      |
|  hubrise_connection/events/   |   +--------------------------+
|  outbox, order_price_changes, |
|  integration_switches/_log    |
+-------------------------------+
```

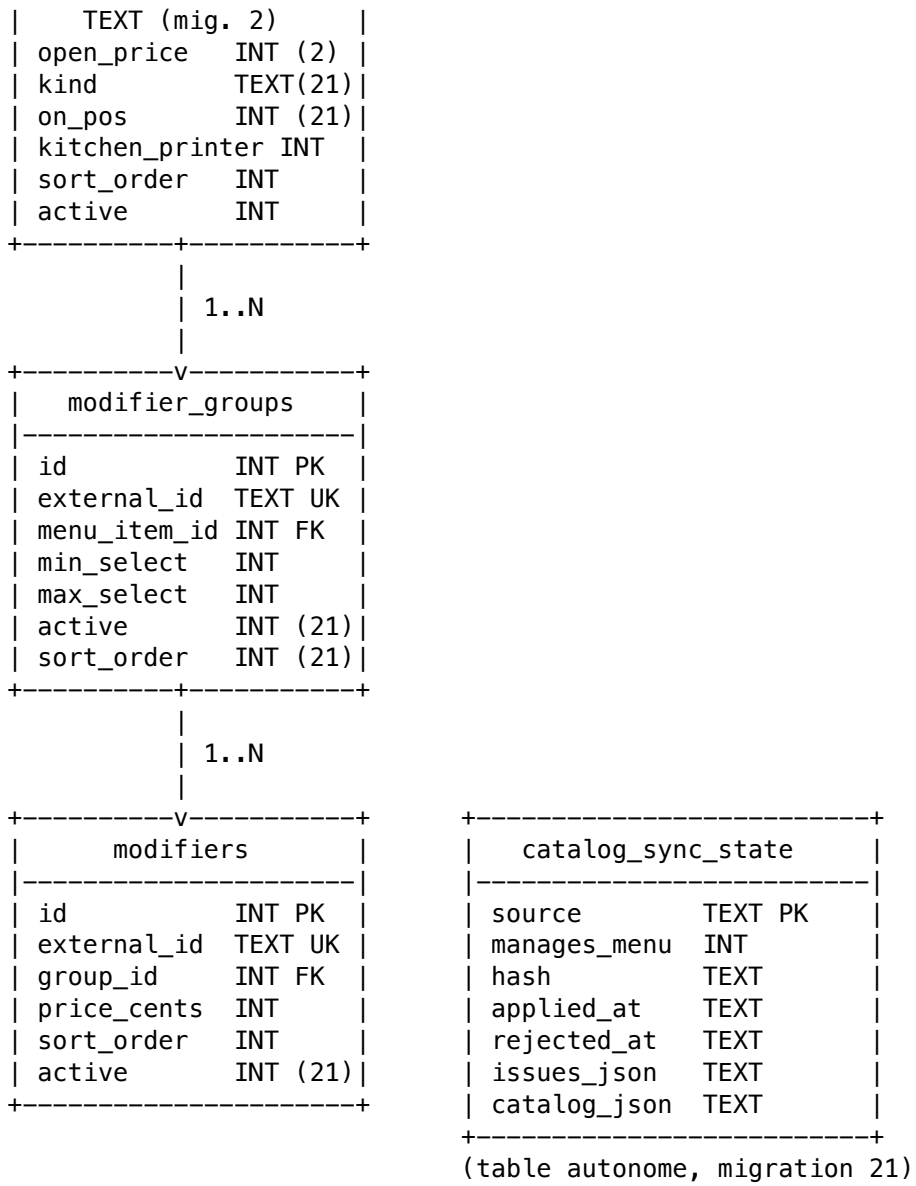
`_migrations_applied` (suivi des migrations) est hors domaine.

---

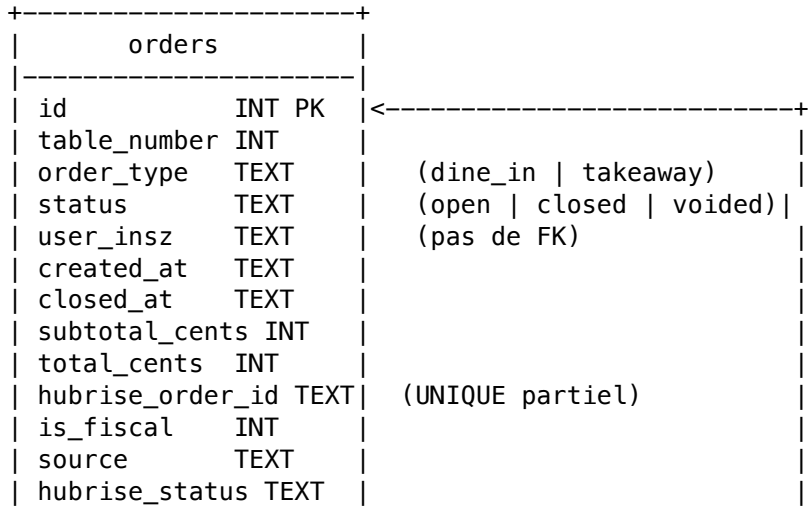
2. Diagrammes ER détaillés

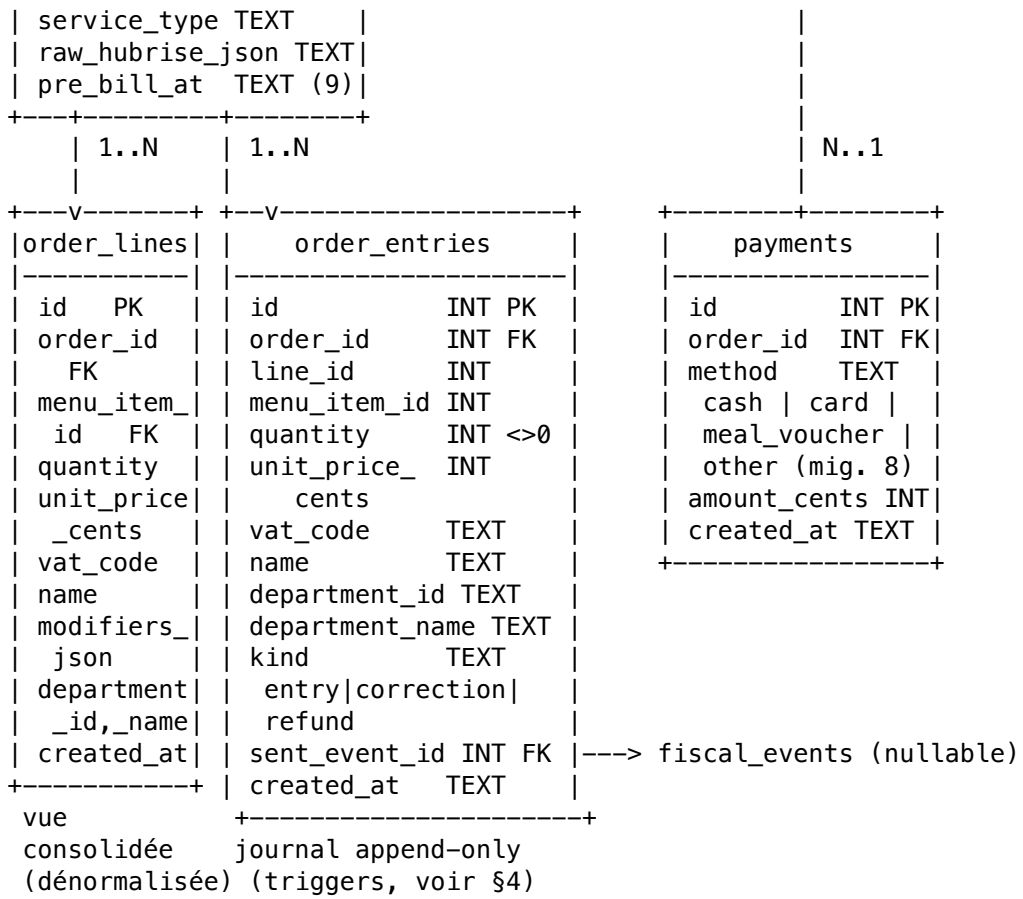
2.1 Catalogue et traductions



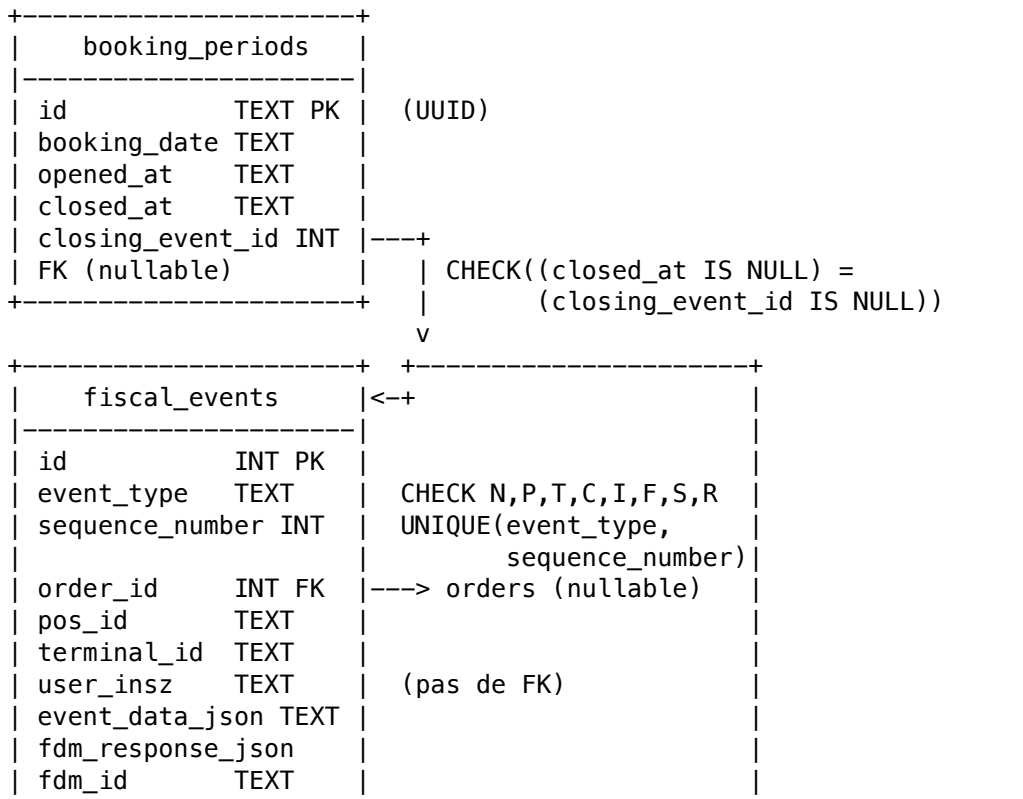


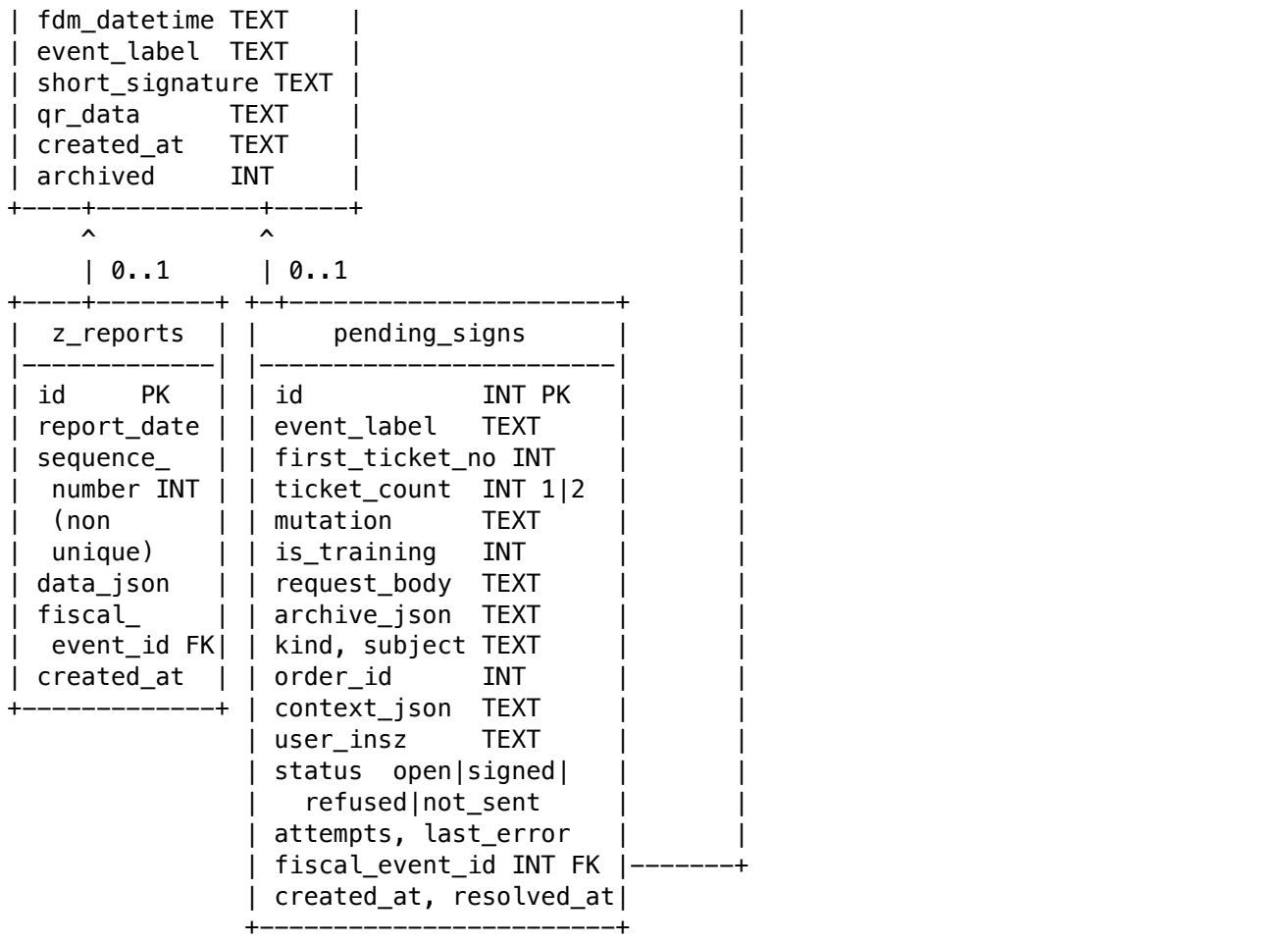
2.2 Commandes, lignes, paiements, journal



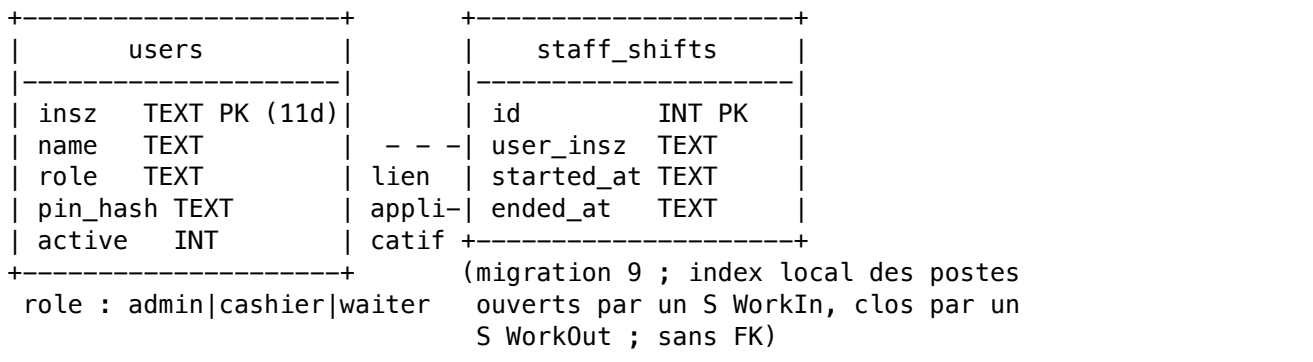


2.3 Fiscal, périodes, signatures en attente, rapports Z

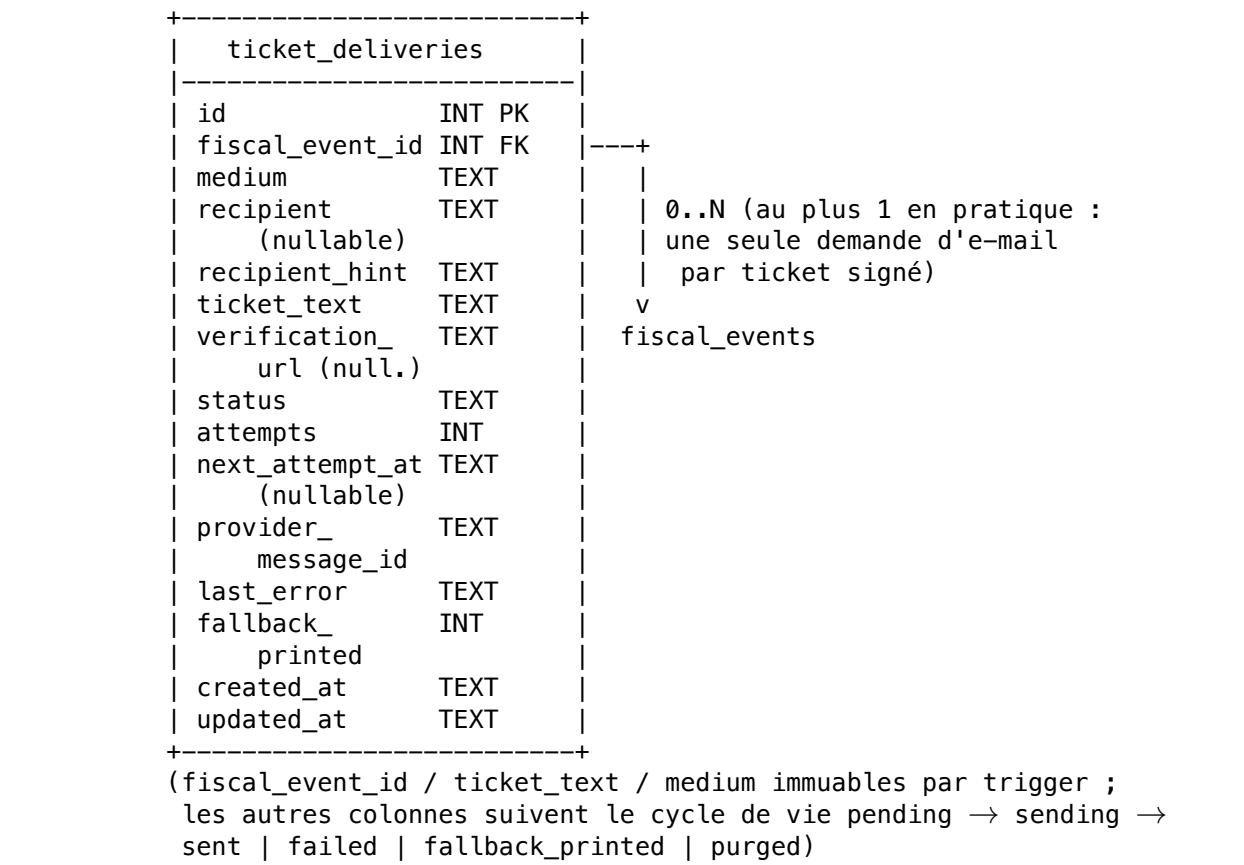




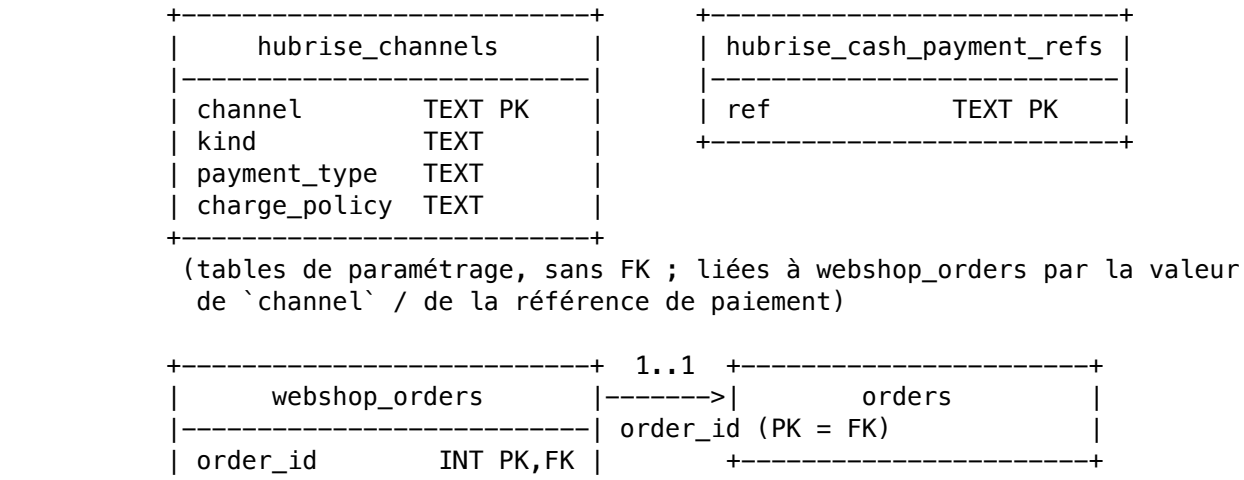
2.4 Utilisateurs et prestations

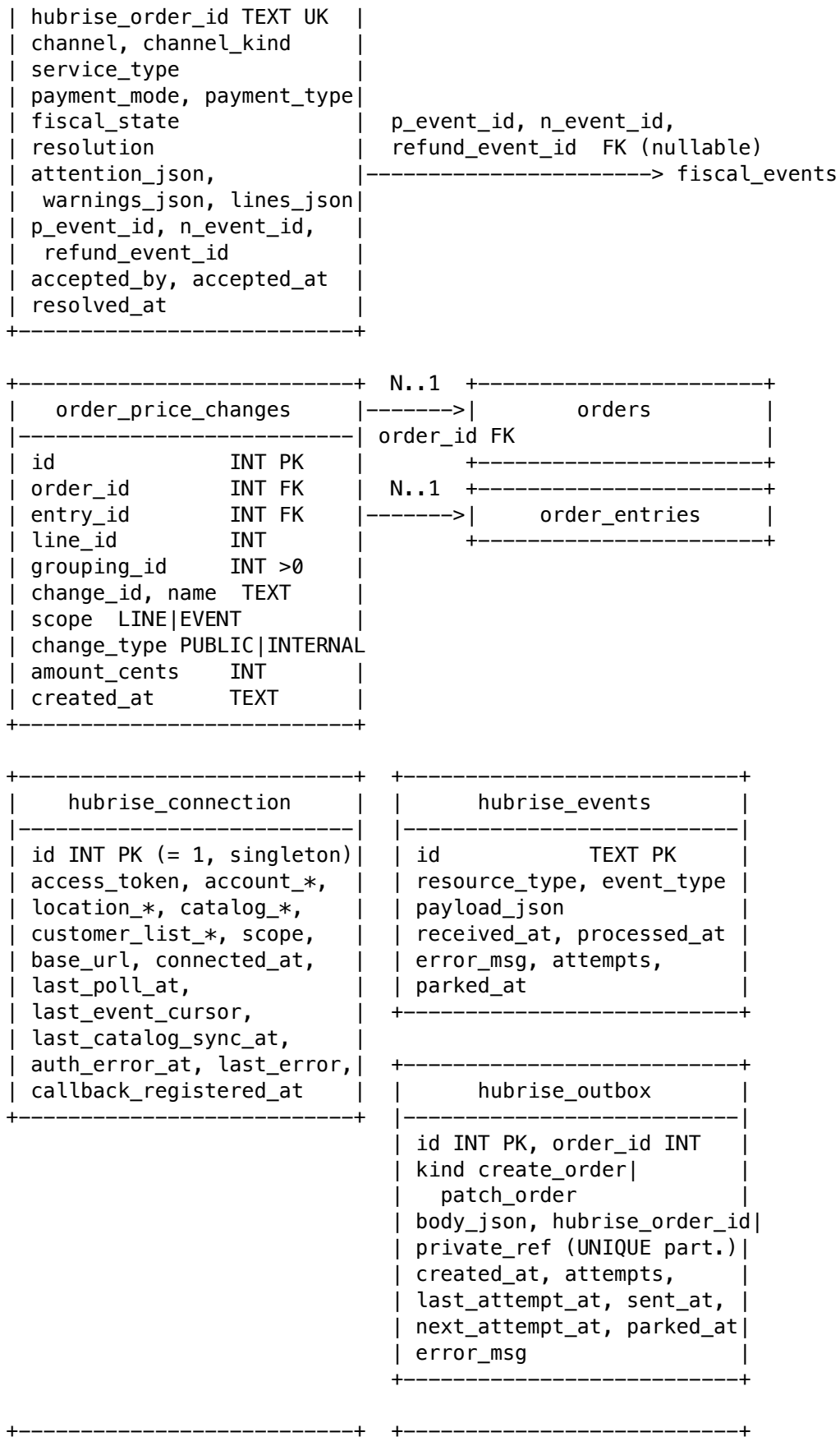


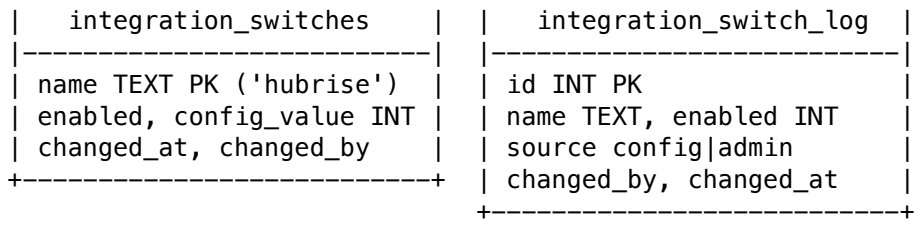
2.5 Ticket numérique et impressions d'original



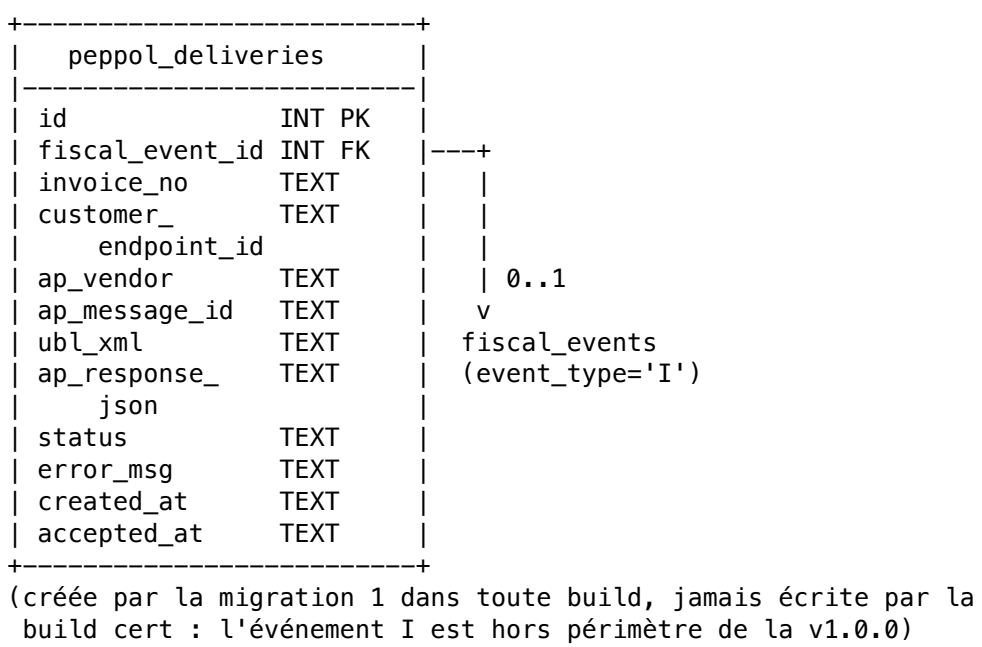
2.6 Commandes en ligne (HubRise, migrations 20 à 25)




| hubrise_order_id TEXT UK |
| channel, channel_kind |
| service_type |
| payment_mode, payment_type|
| fiscal_state |
| resolution |
| attention_json, |
| warnings_json, lines_json|
| p_event_id, n_event_id, |
| refund_event_id |
| accepted_by, accepted_at |
| resolved_at |
p_event_id, n_event_id,
refund_event_id FK (nullable)
fiscal_events
order_price_changes
id INT PK
order_id INT FK
entry_id INT FK
line_id INT
grouping_id INT >0
change_id, name TEXT
scope LINE|EVENT
change_type PUBLIC|INTERNAL
amount_cents INT
created_at TEXT
N..1
orders
order_id FK
N..1
order_entries
hubrise_connection
id INT PK (= 1, singleton)
access_token, account_*,
location_*, catalog_*,
customer_list_*, scope,
base_url, connected_at,
last_poll_at,
last_event_cursor,
last_catalog_sync_at,
auth_error_at, last_error,
callback_registered_at
hubrise_events
id TEXT PK
resource_type, event_type
payload_json
received_at, processed_at
error_msg, attempts,
parked_at
hubrise_outbox
id INT PK, order_id INT
kind create_order|
patch_order
body_json, hubrise_order_id|
private_ref (UNIQUE part.)|
created_at, attempts,
last_attempt_at, sent_at,
next_attempt_at, parked_at|
error_msg



2.7 Peppol (hors binaire certifié v1.0.0)



---

3. Relations principales (verbalisées)

De → Vers	Cardinalité	Sémantique
`categories` → `menu_items`	1 → N	Une catégorie contient N articles de menu
`menu_items` → `modifier_groups`	1 → N	Un article peut avoir N groupes de modificateurs (sauce, cuisson, …)
`modifier_groups` → `modifiers`	1 → N	Chaque groupe contient N options (Soja, Wasabi, …)
`orders` → `order_lines`	1 → N	Une commande contient N lignes
`order_lines` → `menu_items`	N → 1	Référence vers l'article (nom + prix copiés, figés)
`orders` → `order_entries`	1 → N	Le journal chronologique de la commande (ajouts, corrections, remboursements)
`order_entries` → `fiscal_events`	N → 0..1	`sent_event_id` : l'événement P ou N qui a transmis l'entrée
`orders` → `payments`	1 → N	Une commande peut être réglée par N paiements (split, titres-repas)

De → Vers	Cardinalité	Sémantique
`orders` → `fiscal_events`	1 → 0..N	Une commande peut générer 0 (open) à plusieurs events fiscaux (P → N, ou N + C copie, etc.)
`fiscal_events` → `orders`	N → 0..1	Un event fiscal référence 0 (events système comme S, F-WAR) ou 1 commande
`users` → `orders` / `fiscal_events` / `pending_signs` / `staff_shifts`	1 → N	Par la valeur de `user_insz` ; **aucune clé étrangère** (lien applicatif)
`booking_periods` → `fiscal_events`	0..1 → 1	`closing_event_id` : l'événement R qui clôt la période ; un événement ne peut être archivé dans une période close
`pending_signs` → `fiscal_events`	0..1 → 1	`fiscal_event_id` : l'événement archivé (premier événement si `ticket_count = 2`)
`z_reports` → `fiscal_events`	0..1 → 1	Chaque Z archive un event R correspondant. `sequence_number` n'est pas unique (la contrainte d'unicité est portée par `fiscal_events`)
`peppol_deliveries` → `fiscal_events`	0..1 → 1	Liée à un event I ; table jamais écrite par la build cert
`ticket_deliveries` → `fiscal_events`	0..N → 1	Une ligne par ticket dont le support demandé inclut l'e-mail (`DIGITAL`/`PAPER_DIGITAL`) ; ajoutée dans la même transaction que l'event N/T/C correspondant
`original_prints` → `fiscal_events`	0..1 → 1	Au plus une ligne par événement dont l'original a été imprimé (contrainte de clé primaire = garde applicative contre une seconde impression)
`webshop_orders` → `orders`	1 → 1	Une ligne par commande en ligne, à côté de sa ligne `orders`
`webshop_orders` → `fiscal_events`	N → 0..1	`p_event_id`, `n_event_id`, `refund_event_id`
`order_price_changes` → `orders`, `order_entries`	N → 1	Remises d'une commande en ligne, par ligne du journal ; `line_id` désigne la ligne d'`order_lines`
`translations` → (polymorphe)	N → 1	Traduction d'un champ d'une entité (category / menu_item / modifier_group / modifier) dans une langue (FR / NL / DE / EN / ZH)
`translations` → `languages`	N → 1	Référence le code langue
`fiscal_counters` (autonome)	-	Pas de FK ; clé `event_label` ∈ {N, P, T, C, I, F, S, R}

4. Append-only et intégrité

Les tables ci-dessous sont protégées **au niveau de la base** par des triggers SQLite (`RAISE(ABORT, ...)`), en plus de la discipline applicative :

Table	Triggers	Effet
`fiscal_events`	`fiscal_events_no_delete`, `fiscal_events_no_rewrite`	Ni DELETE, ni UPDATE
`fiscal_events`	`fiscal_events_period_open` (BEFORE INSERT)	Refuse un événement dont le `bookingPeriodId` désigne une période close
`z_reports`	`z_reports_no_delete`, `z_reports_no_rewrite`	Ni DELETE, ni UPDATE
`order_entries`	`order_entries_no_delete`, `order_entries_no_rewrite`	Pas de DELETE ; UPDATE interdit sauf `sent_event_id`
`booking_periods`	`booking_periods_no_reopen`, `booking_periods_no_delete`	Une période close ne change plus ; pas de DELETE
`ticket_deliveries`	`ticket_deliveries_no_delete`, `ticket_deliveries_no_rewrite`	Pas de DELETE ; `fiscal_event_id`, `ticket_text`, `medium` immuables (les autres colonnes suivent le cycle de vie de l'envoi : append-only **partiel**)
`original_prints`	`original_prints_no_delete`, `original_prints_no_update`	Ni DELETE, ni UPDATE, aucune exception
`pending_signs`	`pending_signs_request_immutable`, `pending_signs_context_on_resolution`, `pending_signs_no_reopen`, `pending_signs_no_delete`	Ce qui a été envoyé est immuable ; une ligne résolue ne change plus ; pas de DELETE
`order_price_changes`	`order_price_changes_no_delete`, `order_price_changes_no_update`	Ni DELETE, ni UPDATE
`integration_switch_log`	`integration_switch_log_no_update`, `integration_switch_log_no_delete`	Ni DELETE, ni UPDATE

Soit 21 triggers.

Pour les autres tables (catalogue, commandes en cours, paiements, utilisateurs, `staff_shifts`, tables HubRise hors journal) les opérations UPDATE/DELETE sont autorisées via la couche commandes. Les `orders` sont marquées `status='closed'` ou `status='voided'` plutôt que supprimées. Le catalogue synchronisé met à jour ses lignes en place ; l'historique fiscal ne dépend pas de ces lignes (nom et prix sont figés dans `order_lines` / `order_entries`).

La contrainte `UNIQUE(event_type, sequence_number)` de `fiscal_events` empêche tout doublon de compteur fiscal au niveau base, par type d'événement. La table `fiscal_counters` est mise à jour atomiquement dans la même transaction SQLite que l'insert dans `fiscal_events`, sous le verrou de signature unique (cf. *STRATEGIE_COMPTEURS_FR.md* §5). Les requêtes de signature en attente de réponse sont conservées dans `pending_signs` (cf. *SCHEMA_BD_FR.md* §3.9).

---

5. Indexes

Index explicites (hors index implicites des clés primaires et des contraintes UNIQUE) :

Table	Index	Colonnes	Raison
translations	idx_translations_lookup	entity_type, entity_id, lang	Lookup rapide pour le rendu UI multilingue
orders	idx_orders_hubrise_order_id (unique partiel, WHERE hubrise_order_id IS NOT NULL)	hubrise_order_id	Ingestion idempotente des commandes en ligne
order_entries	idx_order_entries_order	order_id, id	Relire le journal d'une commande dans l'ordre
booking_periods	idx_booking_periods_one_open (unique partiel, WHERE closed_at IS NULL)	(closed_at IS NULL)	Au plus une période ouverte
ticket_deliveries	idx_ticket_deliveries_due	status, next_attempt_at	Sélection rapide des envois dus par le worker de réessai (§III.H)
staff_shifts	idx_staff_shifts_open	user_insz, ended_at	Retrouver le poste ouvert d'un employé
pending_signs	idx_pending_signs_one_open (unique partiel, WHERE status = 'open')	(status)	Au plus une requête de signature ouverte
pending_signs	idx_pending_signs_order	order_id, status	Retrouver la requête en attente d'une commande
peppol_deliveries	idx_peppol_deliveries_invoice_no	invoice_no	Recherche par numéro de facture (table inutilisée en v1.0.0)
hubrise_events	idx_hubrise_events_pending (partiel, WHERE processed_at IS NULL)	received_at	Événements HubRise restant à traiter
hubrise_outbox	idx_hubrise_outbox_pending (partiel, WHERE sent_at IS NULL)	created_at	Statuts restant à envoyer
hubrise_outbox	idx_hubrise_outbox_private_ref (unique partiel, WHERE private_ref IS NOT NULL)	private_ref	Une création de commande n'est mise en file qu'une fois
hubrise_outbox	idx_hubrise_outbox_order (partiel, WHERE sent_at IS NULL)	hubrise_order_id, id	Envoi dans l'ordre, par commande
webshop_orders	idx_webshop_orders_state	fiscal_state, resolution	Liste des commandes en ligne par état
order_price_changes	idx_order_price_changes_order	order_id, entry_id	Remises d'une commande

Index implicites utiles : `fiscal_events` (`id`, `UNIQUE(event_type, sequence_number)`), `users` (`insz`), `translations` (`UNIQUE(entity_type, entity_id, field, lang)`), `webshop_orders` (`order_id`, `hubrise_order_id`).

Des indexes additionnels peuvent être ajoutés en post-cert si des problèmes de performance apparaissent à plus grande échelle (multi-restaurant). Pour le périmètre mono-terminal (~150 tickets/jour pour un restaurant type), aucun index supplémentaire n'est nécessaire.

6. Tables des commandes en ligne

Les tables des commandes en ligne (`webshop_orders`, `hubrise_channels`, `hubrise_cash_payment_refs`, `hubrise_outbox`, `hubrise_events`, `hubrise_connection`, `order_price_changes`, `integration_switches`, `integration_switch_log`, `catalog_sync_state`, migrations 20 à 25 et colonnes de la migration 21) sont décrites dans `SCHEMA_BD_FR.md` §5. Elles ne sont écrites que si l'intégration est activée pour l'installation (`COMMANDES_EN_LIGNE_FR.md` §2), sauf le journal de l'interrupteur, qui reçoit au moins la valeur de départ. `webshop_orders.order_id` référence `orders.id` ; ses colonnes `p_event_id`, `n_event_id` et `refund_event_id` référencent `fiscal_events.id`.

---

7. Versionnement du schéma

Le schéma est versionné via `pos-db::migrate::run_migrations`. La table `_migrations_applied` enregistre chaque migration appliquée (`version`, `name`, `applied_at`). Aucune modification de schéma post-cert n'est possible sans une nouvelle demande de certification (Article 109 du manuel).

Version actuelle du schéma : migrations `1` à `10`, puis `20` à `25` (seize migrations ; 11 à 19 sont réservées aux correctifs du binaire certifié) : `initial-schema`, `catalogue-fiscal-fields`, `booking-periods`, `fiscal-archive-append-only`, `ticket-deliveries`, `original-prints`, `german-language`, `meal-voucher-payments`, `shifts-and-pre-bill`, `pending-signs`, `hubrise-transport`, `catalog-sync`, `webshop-orders`, `price-changes`, `platform-charges`, `integration-switch` - liste exacte dans `SCHEMA_BD_FR.md` §6 et `crates/pos-db/src/migrate.rs` (`MIGRATIONS`, `HUBRISE_MIGRATIONS`, `CATALOG_MIGRATIONS`). Les migrations 20 à 25 font partie du binaire certifié.